

Lab Report No 6
Understanding and implementing Artificial Neural
Network
Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

8th Apr, 2025

Department of Computer Engineering,
HITEC University, Taxila

Example Task No 1:

Solution:

Brief description (3-5 lines)

Artificial Neural Network (ANN):

Artificial Neural Network (ANN) is an efficient computing system whose central theme is borrowed from the analogy of biological neural networks. ANNs are also named as “artificial neural systems,” or “parallel distributed processing systems,” or “connectionist systems.” ANN acquires a large collection of units that are interconnected in some pattern to allow communication between the units. These units, also referred to as nodes or neurons, are simple processors which operate in parallel. Every neuron relates to other neuron through a connection link. Each connection link is associated with a weight that has information about the input signal. This is the most useful information for neurons to solve a particular problem because the weight usually excites or inhibits the signal that is being communicated. Each neuron has an internal state, which is called an activation signal. Output signals, which are produced after combining the input signals and activation rule, may be sent to other units.
--

The code

<pre>import numpy as np import pandas as pd from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Dense from sklearn.datasets import load_iris from sklearn.metrics import accuracy_score # Step 1: Load the Iris dataset iris = load_iris() X = iris.data # Features (4 features per sample) y = iris.target # Target classes (3 classes) # Step 2: Preprocess the data (use only two classes for binary classification) # We will classify class 0 (Setosa) vs class 1 (Versicolor) X = X[y != 2] # Remove class 2 (Virginica) y = y[y != 2] # Remove class 2 (Virginica) # Step 3: Split the data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Step 4: Standardize the data (helpful for ANN performance) scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_test = scaler.transform(X_test)</pre>

```

# Step 5: Build the ANN model
model = Sequential()
# Input layer and first hidden layer with 8 neurons and ReLU activation
model.add(Dense(units=8, input_dim=X_train.shape[1], activation='relu'))
# Output layer with 1 neuron and sigmoid activation (for binary classification)
model.add(Dense(units=1, activation='sigmoid'))
# Step 6: Compile the model
model.compile(loss='binary_crossentropy', # For binary classification
optimizer='adam', # Optimizer
metrics=['accuracy']) # Evaluation metric
# Step 7: Train the model
history = model.fit(X_train, y_train, epochs=50, batch_size=10,
validation_split=0.2)
# Step 8: Evaluate the model
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int) # Convert probabilities to binary
predictions
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
# Optional: Plotting training history (loss and accuracy)
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 6))
# Plotting loss
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss during Training')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
# Plotting accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy during Training')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```

The results (Screenshot)

```
Epoch 1/50
7/7 ██████████ 2s 81ms/step - accuracy: 0.9720 - loss: 0.3940 - val_accuracy: 1.0000 - val_loss: 0.3197
Epoch 2/50
7/7 ██████████ 0s 29ms/step - accuracy: 0.9595 - loss: 0.3913 - val_accuracy: 1.0000 - val_loss: 0.3046
Epoch 3/50
7/7 ██████████ 0s 19ms/step - accuracy: 0.9940 - loss: 0.3526 - val_accuracy: 1.0000 - val_loss: 0.2902
Epoch 4/50
7/7 ██████████ 0s 20ms/step - accuracy: 0.9884 - loss: 0.3237 - val_accuracy: 1.0000 - val_loss: 0.2762
Epoch 5/50
7/7 ██████████ 0s 23ms/step - accuracy: 0.9884 - loss: 0.3281 - val_accuracy: 1.0000 - val_loss: 0.2626
Epoch 6/50
7/7 ██████████ 0s 19ms/step - accuracy: 0.9842 - loss: 0.3313 - val_accuracy: 1.0000 - val_loss: 0.2497
Epoch 7/50
7/7 ██████████ 0s 25ms/step - accuracy: 0.9940 - loss: 0.3017 - val_accuracy: 1.0000 - val_loss: 0.2376
Epoch 8/50
7/7 ██████████ 0s 19ms/step - accuracy: 0.9780 - loss: 0.2908 - val_accuracy: 1.0000 - val_loss: 0.2259
Epoch 9/50
7/7 ██████████ 0s 20ms/step - accuracy: 0.9780 - loss: 0.2681 - val_accuracy: 1.0000 - val_loss: 0.2146
Epoch 10/50
7/7 ██████████ 0s 27ms/step - accuracy: 1.0000 - loss: 0.2601 - val_accuracy: 1.0000 - val_loss: 0.2040
Epoch 11/50
7/7 ██████████ 0s 23ms/step - accuracy: 1.0000 - loss: 0.2610 - val_accuracy: 1.0000 - val_loss: 0.1936
Epoch 12/50
7/7 ██████████ 0s 20ms/step - accuracy: 1.0000 - loss: 0.2401 - val_accuracy: 1.0000 - val_loss: 0.1836
Epoch 13/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2186 - val_accuracy: 1.0000 - val_loss: 0.1743
Epoch 14/50
7/7 ██████████ 0s 19ms/step - accuracy: 1.0000 - loss: 0.2201 - val_accuracy: 1.0000 - val_loss: 0.1655
Epoch 15/50
7/7 ██████████ 0s 23ms/step - accuracy: 1.0000 - loss: 0.2152 - val_accuracy: 1.0000 - val_loss: 0.1570
Epoch 16/50
7/7 ██████████ 0s 23ms/step - accuracy: 1.0000 - loss: 0.2024 - val_accuracy: 1.0000 - val_loss: 0.1489
Epoch 17/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.1803 - val_accuracy: 1.0000 - val_loss: 0.1414
Epoch 18/50
```

```

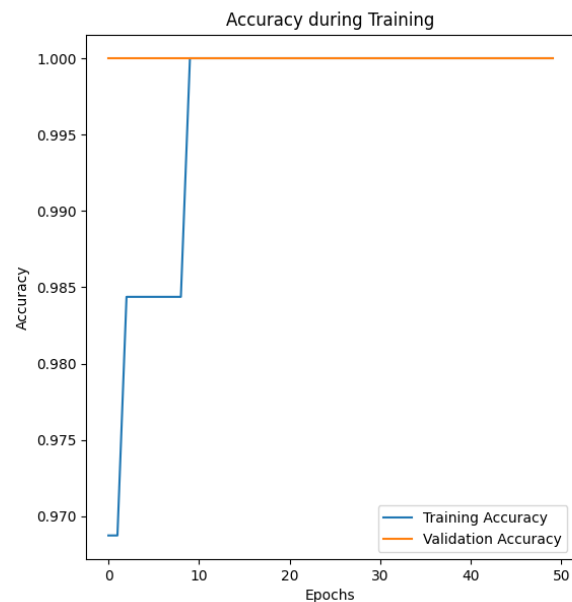
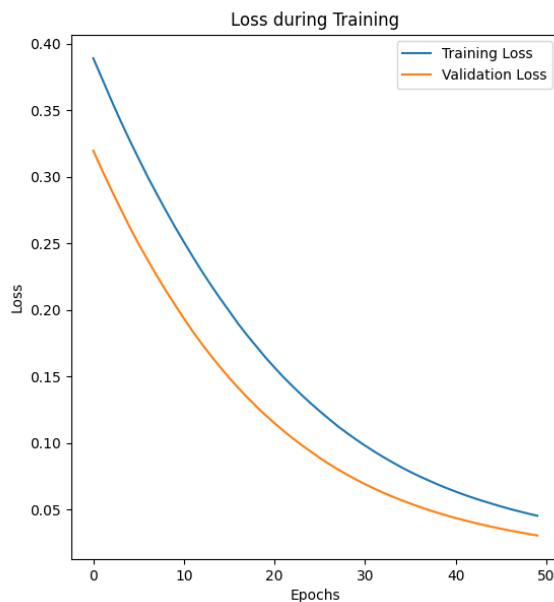
7/7 ██████████ 0s 20ms/step - accuracy: 1.0000 - loss: 0.1700 - val_accuracy: 1.0000 - val
_loss: 0.1343
Epoch 19/50
7/7 ██████████ 0s 25ms/step - accuracy: 1.0000 - loss: 0.1842 - val_accuracy: 1.0000 - val
_loss: 0.1274
Epoch 20/50
7/7 ██████████ 0s 23ms/step - accuracy: 1.0000 - loss: 0.1590 - val_accuracy: 1.0000 - val
_loss: 0.1210
Epoch 21/50
7/7 ██████████ 0s 25ms/step - accuracy: 1.0000 - loss: 0.1651 - val_accuracy: 1.0000 - val
_loss: 0.1149
Epoch 22/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.1475 - val_accuracy: 1.0000 - val
_loss: 0.1091
Epoch 23/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.1495 - val_accuracy: 1.0000 - val
_loss: 0.1036
Epoch 24/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.1386 - val_accuracy: 1.0000 - val
_loss: 0.0984
Epoch 25/50
7/7 ██████████ 0s 23ms/step - accuracy: 1.0000 - loss: 0.1360 - val_accuracy: 1.0000 - val
_loss: 0.0935
Epoch 26/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.1295 - val_accuracy: 1.0000 - val
_loss: 0.0888
Epoch 27/50
7/7 ██████████ 0s 25ms/step - accuracy: 1.0000 - loss: 0.1269 - val_accuracy: 1.0000 - val
_loss: 0.0843
Epoch 28/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.1070 - val_accuracy: 1.0000 - val
_loss: 0.0802
Epoch 29/50
7/7 ██████████ 0s 25ms/step - accuracy: 1.0000 - loss: 0.1092 - val_accuracy: 1.0000 - val
_loss: 0.0762
Epoch 30/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.1036 - val_accuracy: 1.0000 - val
_loss: 0.0725
Epoch 31/50
7/7 ██████████ 0s 17ms/step - accuracy: 1.0000 - loss: 0.0972 - val_accuracy: 1.0000 - val
_loss: 0.0690
Epoch 32/50
7/7 ██████████ 0s 23ms/step - accuracy: 1.0000 - loss: 0.0973 - val_accuracy: 1.0000 - val
_loss: 0.0657
Epoch 33/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.0959 - val_accuracy: 1.0000 - val
_loss: 0.0626
Epoch 34/50
7/7 ██████████ 0s 25ms/step - accuracy: 1.0000 - loss: 0.0888 - val_accuracy: 1.0000 - val
_loss: 0.0597
Epoch 35/50
7/7 ██████████ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0856 - val_accuracy: 1.0000 - val
_loss: 0.0570
Epoch 36/50
7/7 ██████████ 0s 20ms/step - accuracy: 1.0000 - loss: 0.0723 - val_accuracy: 1.0000 - val
_loss: 0.0545
Epoch 37/50

```

```

7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0704 - val_accuracy: 1.0000 - val
_loss: 0.0520
Epoch 38/50
7/7 ██████████ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0760 - val_accuracy: 1.0000 - val
_loss: 0.0497
Epoch 39/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.0722 - val_accuracy: 1.0000 - val
_loss: 0.0475
Epoch 40/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0708 - val_accuracy: 1.0000 - val
_loss: 0.0455
Epoch 41/50
7/7 ██████████ 0s 20ms/step - accuracy: 1.0000 - loss: 0.0644 - val_accuracy: 1.0000 - val
_loss: 0.0436
Epoch 42/50
7/7 ██████████ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0587 - val_accuracy: 1.0000 - val
_loss: 0.0418
Epoch 43/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0556 - val_accuracy: 1.0000 - val
_loss: 0.0401
Epoch 44/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.0547 - val_accuracy: 1.0000 - val
_loss: 0.0385
Epoch 45/50
7/7 ██████████ 0s 20ms/step - accuracy: 1.0000 - loss: 0.0598 - val_accuracy: 1.0000 - val
_loss: 0.0370
Epoch 46/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0558 - val_accuracy: 1.0000 - val
_loss: 0.0355
Epoch 47/50
7/7 ██████████ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0494 - val_accuracy: 1.0000 - val
_loss: 0.0342
Epoch 48/50
7/7 ██████████ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0502 - val_accuracy: 1.0000 - val_loss: 0.0329
Epoch 49/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.0410 - val_accuracy: 1.0000 - val_loss: 0.0317
Epoch 50/50
7/7 ██████████ 0s 22ms/step - accuracy: 1.0000 - loss: 0.0450 - val_accuracy: 1.0000 - val_loss: 0.0305
1/1 ██████████ 0s 112ms/step
Model Accuracy: 100.00%

```



Lab Task No 1(i):

Solution:

Brief description (3-5 lines)
Build a Simple ANN for Binary Classification: Use a binary classification dataset (e.g., Iris dataset, classify Setosa vs. Virginica).
The code
<pre>import numpy as np import pandas as pd import matplotlib.pyplot as plt from sklearn import datasets from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.metrics import classification_report from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Dense from tensorflow.keras.optimizers import Adam # Load the Iris dataset iris = datasets.load_iris() X = iris.data y = iris.target # Filter only Setosa (0) and Virginica (2) binary_filter = (y == 0) (y == 2) X = X[binary_filter] y = y[binary_filter] # Convert labels to 0 and 1 y = (y == 2).astype(int) # Setosa = 0, Virginica = 1 # Train-test split X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Feature scaling scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # Build ANN model model = Sequential() model.add(Dense(10, input_shape=(4,), activation='relu')) model.add(Dense(1, activation='sigmoid'))</pre>

```

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.01),
loss='binary_crossentropy', metrics=['accuracy'])
# Train the model
history = model.fit(X_train_scaled, y_train, epochs=50,
validation_data=(X_test_scaled, y_test), verbose=0)
# Evaluate the model
loss, accuracy = model.evaluate(X_test_scaled, y_test)
print(f'Test Accuracy: {accuracy:.2f}')
# Classification report
y_pred = (model.predict(X_test_scaled) > 0.5).astype("int32")
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# Plot training history
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

The results (Screenshot)

```

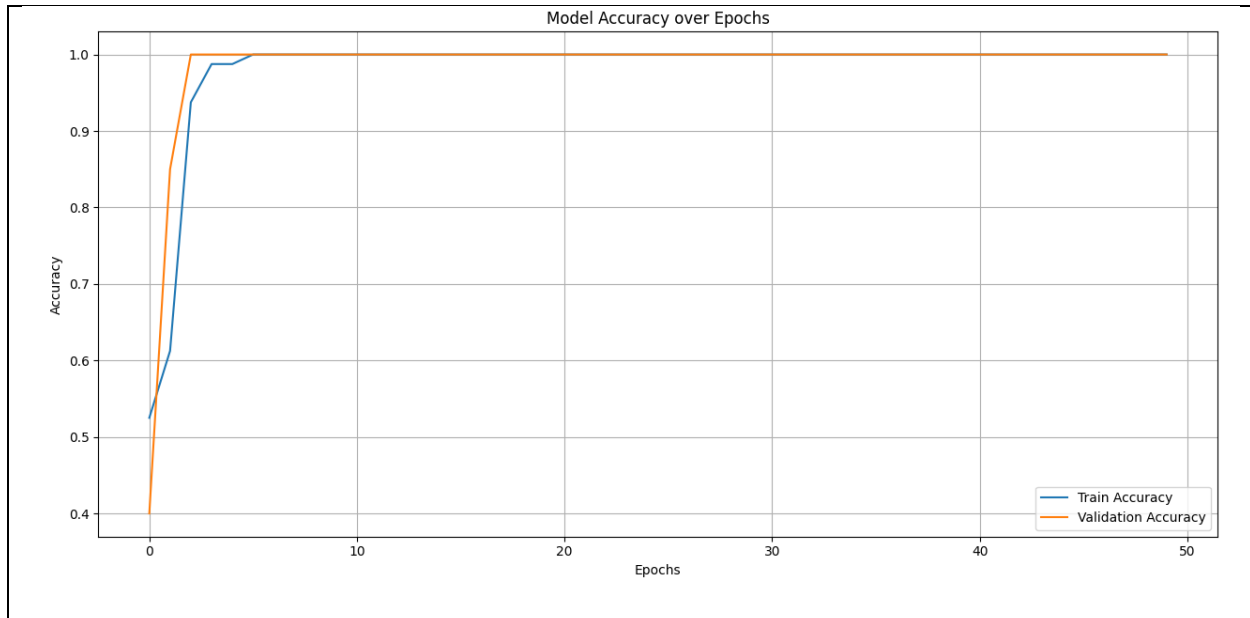
1/1 ————— 0s 82ms/step - accuracy: 1.0000 - loss: 0.0026
Test Accuracy: 1.00
1/1 ————— 0s 125ms/step

Classification Report:
              precision    recall  f1-score   support

         0           1.00      1.00      1.00         12
         1           1.00      1.00      1.00          8

   accuracy                1.00                1.00         20
  macro avg              1.00              1.00              1.00         20
 weighted avg              1.00              1.00              1.00         20

```

Lab Task No 1(ii):

Solution:

Brief description (3-5 lines)

Build a Simple ANN for Binary Classification

Build an ANN with:

- Input Layer: 4 neurons (one for each feature).
- Hidden Layer: 8 neurons with ReLU activation.
- Output Layer: 1 neuron with sigmoid activation for binary classification.

The code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import plot_model
import tensorflow as tf

# 1. Generate a synthetic binary classification dataset
X, y = make_classification(n_samples=1000, n_features=4, n_informative=3,
n_redundant=0, n_clusters_per_class=1, random_state=42)

# 2. Split into training and testing data
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# 3. Standardize the feature values
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# 4. Build the ANN model
model = Sequential([
    Dense(8, input_shape=(4,), activation='relu'), # Hidden Layer
    Dense(1, activation='sigmoid')                # Output Layer
])
# 5. Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# 6. Train the model
history = model.fit(X_train, y_train, epochs=50, batch_size=16,
validation_split=0.2)
# 7. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f'\nTest Accuracy: {accuracy:.4f}')
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

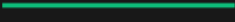
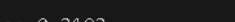
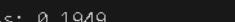
```

```
plot_model(model, to_file='model_architecture.png', show_shapes=True,
show_layer_names=True)
```

The results (Screenshot)

```
Epoch 1/50
40/40 ————— 5s 30ms/step - accuracy: 0.5722 - loss: 0.6638 - val_accuracy: 0.6562 - v
al_loss: 0.6555
Epoch 2/50
40/40 ————— 0s 6ms/step - accuracy: 0.6289 - loss: 0.6430 - val_accuracy: 0.7063 - va
l_loss: 0.6100
Epoch 3/50
40/40 ————— 0s 7ms/step - accuracy: 0.7263 - loss: 0.5635 - val_accuracy: 0.7312 - va
l_loss: 0.5680
Epoch 4/50
40/40 ————— 0s 4ms/step - accuracy: 0.7084 - loss: 0.5534 - val_accuracy: 0.7812 - va
l_loss: 0.5278
Epoch 5/50
40/40 ————— 0s 4ms/step - accuracy: 0.7805 - loss: 0.5135 - val_accuracy: 0.8188 - va
l_loss: 0.4878
Epoch 6/50
40/40 ————— 0s 5ms/step - accuracy: 0.8150 - loss: 0.4774 - val_accuracy: 0.8438 - va
l_loss: 0.4503
Epoch 7/50
40/40 ————— 0s 7ms/step - accuracy: 0.8342 - loss: 0.4495 - val_accuracy: 0.8625 - va
l_loss: 0.4141
Epoch 8/50
40/40 ————— 0s 7ms/step - accuracy: 0.8653 - loss: 0.4077 - val_accuracy: 0.8813 - va
l_loss: 0.3793
Epoch 9/50
40/40 ————— 0s 5ms/step - accuracy: 0.8967 - loss: 0.3646 - val_accuracy: 0.9000 - va
l_loss: 0.3482
```

```

Epoch 10/50
40/40  0s 7ms/step - accuracy: 0.8695 - loss: 0.3507 - val_accuracy: 0.9312 - val_loss: 0.3193
Epoch 11/50
40/40  0s 10ms/step - accuracy: 0.9055 - loss: 0.3211 - val_accuracy: 0.9375 - val_loss: 0.2930
Epoch 12/50
40/40  0s 6ms/step - accuracy: 0.9261 - loss: 0.2969 - val_accuracy: 0.9438 - val_loss: 0.2707
Epoch 13/50
40/40  0s 4ms/step - accuracy: 0.8980 - loss: 0.2981 - val_accuracy: 0.9438 - val_loss: 0.2502
Epoch 14/50
40/40  0s 7ms/step - accuracy: 0.9233 - loss: 0.2668 - val_accuracy: 0.9500 - val_loss: 0.2335
Epoch 15/50
40/40  1s 13ms/step - accuracy: 0.9260 - loss: 0.2596 - val_accuracy: 0.9625 - val_loss: 0.2192
Epoch 16/50
40/40  0s 11ms/step - accuracy: 0.9332 - loss: 0.2522 - val_accuracy: 0.9688 - val_loss: 0.2066
Epoch 17/50
40/40  0s 10ms/step - accuracy: 0.9386 - loss: 0.2128 - val_accuracy: 0.9688 - val_loss: 0.1949
Epoch 18/50
40/40  0s 5ms/step - accuracy: 0.9411 - loss: 0.2108 - val_accuracy: 0.9625 - val_loss: 0.1851
Epoch 19/50
40/40  0s 6ms/step - accuracy: 0.9388 - loss: 0.1994 - val_accuracy: 0.9625 - val_loss: 0.1772
Epoch 20/50
40/40  0s 7ms/step - accuracy: 0.9379 - loss: 0.2034 - val_accuracy: 0.9688 - val_loss: 0.1700
Epoch 21/50
40/40  0s 6ms/step - accuracy: 0.9460 - loss: 0.1905 - val_accuracy: 0.9688 - val_loss: 0.1635
Epoch 22/50
40/40  0s 6ms/step - accuracy: 0.9351 - loss: 0.1943 - val_accuracy: 0.9688 - val_loss: 0.1579
Epoch 23/50
40/40  0s 5ms/step - accuracy: 0.9303 - loss: 0.1846 - val_accuracy: 0.9688 - val_loss: 0.1532
Epoch 24/50
40/40  0s 7ms/step - accuracy: 0.9453 - loss: 0.1689 - val_accuracy: 0.9688 - val_loss: 0.1483
Epoch 25/50
40/40  0s 8ms/step - accuracy: 0.9464 - loss: 0.1615 - val_accuracy: 0.9688 - val_loss: 0.1444
Epoch 26/50
40/40  1s 6ms/step - accuracy: 0.9459 - loss: 0.1619 - val_accuracy: 0.9688 - val_loss: 0.1412
Epoch 27/50

```

```

40/40 ————— 0s 5ms/step - accuracy: 0.9543 - loss: 0.1399 - val_accuracy: 0.9688 - va
l_loss: 0.1378
Epoch 28/50
40/40 ————— 0s 5ms/step - accuracy: 0.9471 - loss: 0.1879 - val_accuracy: 0.9688 - va
l_loss: 0.1349
Epoch 29/50
40/40 ————— 0s 5ms/step - accuracy: 0.9502 - loss: 0.1476 - val_accuracy: 0.9688 - va
l_loss: 0.1320
Epoch 30/50
40/40 ————— 0s 5ms/step - accuracy: 0.9523 - loss: 0.1425 - val_accuracy: 0.9688 - va
l_loss: 0.1295
Epoch 31/50
40/40 ————— 0s 5ms/step - accuracy: 0.9393 - loss: 0.1683 - val_accuracy: 0.9688 - va
l_loss: 0.1270
Epoch 32/50
40/40 ————— 0s 5ms/step - accuracy: 0.9475 - loss: 0.1443 - val_accuracy: 0.9750 - va
l_loss: 0.1246
Epoch 33/50
40/40 ————— 0s 5ms/step - accuracy: 0.9276 - loss: 0.1749 - val_accuracy: 0.9750 - va
l_loss: 0.1227
Epoch 34/50
40/40 ————— 0s 5ms/step - accuracy: 0.9483 - loss: 0.1666 - val_accuracy: 0.9750 - va
l_loss: 0.1205
Epoch 35/50
40/40 ————— 0s 5ms/step - accuracy: 0.9481 - loss: 0.1483 - val_accuracy: 0.9750 - va
l_loss: 0.1188
Epoch 36/50
40/40 ————— 0s 5ms/step - accuracy: 0.9537 - loss: 0.1415 - val_accuracy: 0.9750 - va
l_loss: 0.1174
Epoch 37/50
40/40 ————— 0s 7ms/step - accuracy: 0.9543 - loss: 0.1531 - val_accuracy: 0.9750 - va
l_loss: 0.1152
Epoch 38/50
40/40 ————— 0s 5ms/step - accuracy: 0.9677 - loss: 0.1136 - val_accuracy: 0.9750 - va
l_loss: 0.1137
Epoch 39/50
40/40 ————— 0s 7ms/step - accuracy: 0.9668 - loss: 0.1329 - val_accuracy: 0.9688 - va
l_loss: 0.1123
Epoch 40/50
40/40 ————— 0s 9ms/step - accuracy: 0.9570 - loss: 0.1329 - val_accuracy: 0.9688 - va
l_loss: 0.1110
Epoch 41/50
40/40 ————— 0s 11ms/step - accuracy: 0.9682 - loss: 0.1117 - val_accuracy: 0.9688 - v
al_loss: 0.1093
Epoch 42/50
40/40 ————— 0s 9ms/step - accuracy: 0.9480 - loss: 0.1439 - val_accuracy: 0.9688 - va
l_loss: 0.1081
Epoch 43/50
40/40 ————— 1s 13ms/step - accuracy: 0.9605 - loss: 0.1130 - val_accuracy: 0.9688 - v
al_loss: 0.1070
Epoch 44/50
40/40 ————— 0s 8ms/step - accuracy: 0.9581 - loss: 0.1357 - val_accuracy: 0.9688 - va
l_loss: 0.1056
Epoch 45/50

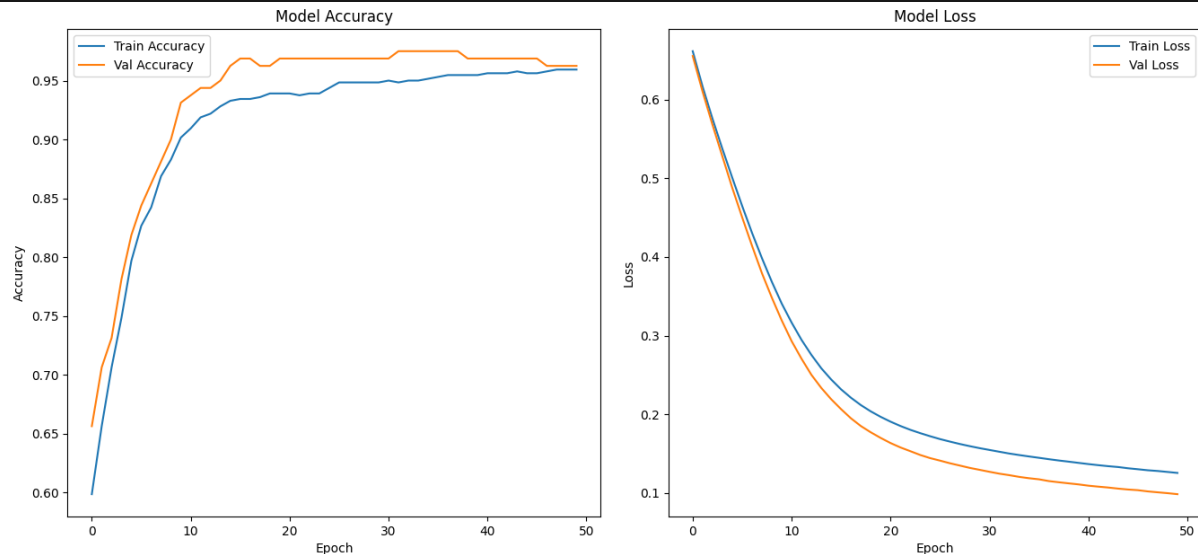
```

```

40/40 ————— 1s 13ms/step - accuracy: 0.9503 - loss: 0.1477 - val_accuracy: 0.9688 - v
al_loss: 0.1045
Epoch 46/50
40/40 ————— 0s 8ms/step - accuracy: 0.9630 - loss: 0.1199 - val_accuracy: 0.9688 - va
l_loss: 0.1036
Epoch 47/50
40/40 ————— 1s 11ms/step - accuracy: 0.9607 - loss: 0.1240 - val_accuracy: 0.9625 - v
al_loss: 0.1021
Epoch 48/50
40/40 ————— 0s 7ms/step - accuracy: 0.9513 - loss: 0.1503 - val_accuracy: 0.9625 - val_loss: 0.1009
Epoch 49/50
40/40 ————— 1s 11ms/step - accuracy: 0.9505 - loss: 0.1399 - val_accuracy: 0.9625 - val_loss: 0.0998
Epoch 50/50
40/40 ————— 0s 8ms/step - accuracy: 0.9636 - loss: 0.1154 - val_accuracy: 0.9625 - val_loss: 0.0985
7/7 ————— 1s 15ms/step - accuracy: 0.9162 - loss: 0.2463

Test Accuracy: 0.9150

```



Lab Task No 1(iii):

Solution:

Brief description (3-5 lines)

Build a Simple ANN for Binary Classification

Use binary cross-entropy as the loss function and adam optimizer.

The code

```

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
# Step 1: Generate Synthetic Binary Classification Data

```

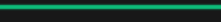
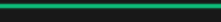
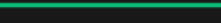
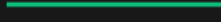
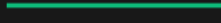
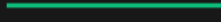
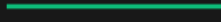
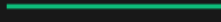
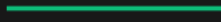
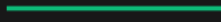
```

X, y = make_classification(n_samples=1000, n_features=10,
n_classes=2, n_informative=5, n_redundant=2, random_state=1)
# Step 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Step 3: Build ANN Model
model = Sequential([
    Dense(16, activation='relu', input_shape=(X.shape[1],)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid') # Output layer for binary classification
])
# Step 4: Compile the model
model.compile(optimizer=Adam(), loss='binary_crossentropy',
metrics=['accuracy'])
# Step 5: Train the model
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=30,
batch_size=32, verbose=1)
# Step 6: Plot accuracy and loss
plt.figure(figsize=(12, 5))
# Accuracy Plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', marker='x')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
# Loss Plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', marker='o')
plt.plot(history.history['val_loss'], label='Validation Loss', marker='x')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```

The results (Screenshot)

```

Epoch 1/30
25/25  3s 21ms/step - accuracy: 0.5233 - loss: 0.7550 - val_accuracy: 0.4600 - v
al_loss: 0.7655
Epoch 2/30
25/25  0s 7ms/step - accuracy: 0.5296 - loss: 0.7065 - val_accuracy: 0.4750 - va
l_loss: 0.7154
Epoch 3/30
25/25  0s 9ms/step - accuracy: 0.5828 - loss: 0.6611 - val_accuracy: 0.5150 - va
l_loss: 0.6771
Epoch 4/30
25/25  0s 7ms/step - accuracy: 0.5725 - loss: 0.6444 - val_accuracy: 0.5750 - va
l_loss: 0.6385
Epoch 5/30
25/25  0s 7ms/step - accuracy: 0.6694 - loss: 0.5976 - val_accuracy: 0.6100 - va
l_loss: 0.6034
Epoch 6/30
25/25  0s 10ms/step - accuracy: 0.7246 - loss: 0.5469 - val_accuracy: 0.6700 - v
al_loss: 0.5656
Epoch 7/30
25/25  0s 10ms/step - accuracy: 0.7371 - loss: 0.5241 - val_accuracy: 0.7150 - v
al_loss: 0.5256
Epoch 8/30
25/25  0s 10ms/step - accuracy: 0.7902 - loss: 0.4697 - val_accuracy: 0.7350 - v
al_loss: 0.4892
Epoch 9/30
25/25  0s 11ms/step - accuracy: 0.7986 - loss: 0.4467 - val_accuracy: 0.7800 - v
al_loss: 0.4636
Epoch 10/30
25/25  0s 13ms/step - accuracy: 0.8267 - loss: 0.4119 - val_accuracy: 0.7950 - v
al_loss: 0.4351
Epoch 11/30
25/25  1s 15ms/step - accuracy: 0.8261 - loss: 0.3911 - val_accuracy: 0.8000 - v
al_loss: 0.4187
Epoch 12/30
25/25  0s 15ms/step - accuracy: 0.8378 - loss: 0.3868 - val_accuracy: 0.8050 - v
al_loss: 0.4024
Epoch 13/30
25/25  0s 12ms/step - accuracy: 0.8500 - loss: 0.3560 - val_accuracy: 0.8250 - v
al_loss: 0.3894
Epoch 14/30
25/25  0s 12ms/step - accuracy: 0.8594 - loss: 0.3398 - val_accuracy: 0.8300 - v
al_loss: 0.3801
Epoch 15/30
25/25  0s 9ms/step - accuracy: 0.8742 - loss: 0.3229 - val_accuracy: 0.8350 - va

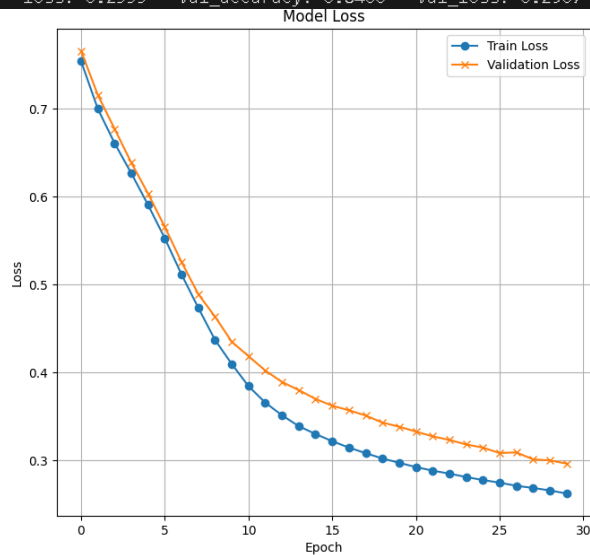
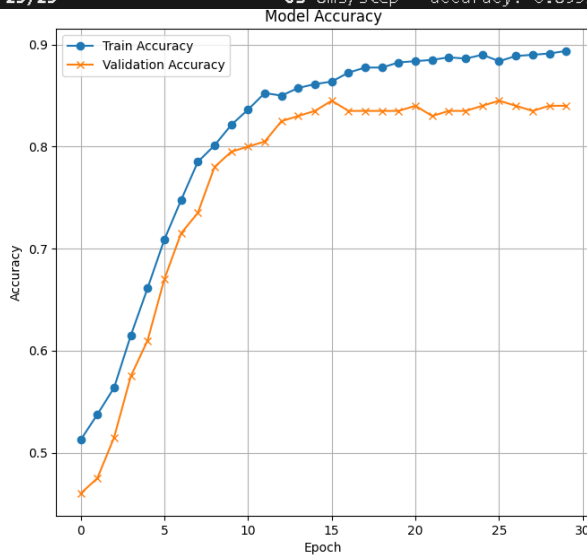
```



```

l_loss: 0.3702
Epoch 16/30
25/25 — 0s 14ms/step - accuracy: 0.8718 - loss: 0.3376 - val_accuracy: 0.8450 - v
al_loss: 0.3624
Epoch 17/30
25/25 — 0s 10ms/step - accuracy: 0.8665 - loss: 0.3161 - val_accuracy: 0.8350 - v
al_loss: 0.3572
Epoch 18/30
25/25 — 0s 14ms/step - accuracy: 0.8826 - loss: 0.2819 - val_accuracy: 0.8350 - v
al_loss: 0.3512
Epoch 19/30
25/25 — 0s 7ms/step - accuracy: 0.8828 - loss: 0.3085 - val_accuracy: 0.8350 - va
l_loss: 0.3432
Epoch 20/30
25/25 — 0s 8ms/step - accuracy: 0.8711 - loss: 0.3280 - val_accuracy: 0.8350 - va
l_loss: 0.3385
Epoch 21/30
25/25 — 0s 10ms/step - accuracy: 0.8740 - loss: 0.3097 - val_accuracy: 0.8400 - v
al_loss: 0.3330
Epoch 22/30
25/25 — 0s 12ms/step - accuracy: 0.8917 - loss: 0.2741 - val_accuracy: 0.8300 - v
al_loss: 0.3275
Epoch 23/30
25/25 — 0s 10ms/step - accuracy: 0.8743 - loss: 0.3001 - val_accuracy: 0.8350 - v
al_loss: 0.3235
Epoch 24/30
25/25 — 0s 10ms/step - accuracy: 0.9078 - loss: 0.2614 - val_accuracy: 0.8350 - v
al_loss: 0.3184
Epoch 25/30
25/25 — 0s 7ms/step - accuracy: 0.8855 - loss: 0.2915 - val_accuracy: 0.8400 - val_loss: 0.3148
Epoch 26/30
25/25 — 0s 7ms/step - accuracy: 0.8920 - loss: 0.2722 - val_accuracy: 0.8450 - val_loss: 0.3086
Epoch 27/30
25/25 — 0s 15ms/step - accuracy: 0.8933 - loss: 0.2639 - val_accuracy: 0.8400 - val_loss: 0.3093
Epoch 28/30
25/25 — 0s 14ms/step - accuracy: 0.8915 - loss: 0.2803 - val_accuracy: 0.8350 - val_loss: 0.3014
Epoch 29/30
25/25 — 0s 6ms/step - accuracy: 0.8926 - loss: 0.2610 - val_accuracy: 0.8400 - val_loss: 0.3002
Epoch 30/30
25/25 — 0s 8ms/step - accuracy: 0.8932 - loss: 0.2555 - val_accuracy: 0.8400 - val_loss: 0.2967

```



Conclusion

In conclusion, Artificial Neural Networks (ANNs) are crucial in modern AI applications due to their adaptability and accuracy. They mimic human learning,

enhance decision-making across domains, and enable engineers to innovate intelligent systems for real-world challenges, making them essential in AI applications.